

Guia da disciplina

Programação Orientada a Objetos II

v0.2

Evandro J.R. Silva
2026.2

Lista de Figuras

Lista de Tabelas

Sumário

Lista de Figuras	i
Lista de Tabelas	i
Sumário	i
I Plano de Ensino	1
1 PPC	3
1.1 Ementa	3
1.2 Bibliografia	3
1.2.1 Básica	3
1.2.2 Complementar	4
2 Informações atualizadas	5
2.1 Planos de ensino anteriores	6
2.2 Novo Plano de Ensino	7

2.2.1	Novos subtópicos da ementa	7
2.2.2	Novos objetivos	9
2.2.3	Nova bibliografia	10
2.2.3.1	Interfaces e processamento de eventos	10
2.2.3.2	Programação gráfica	10
2.2.3.3	Manipulação de arquivos	11
2.2.3.4	Programação concorrente com múltiplas threads	12
2.2.3.5	Programação em rede	12
2.2.3.6	Conectividade com bancos de dados	13
Referências		15

Parte I

Plano de Ensino

Capítulo 1

PPC

Neste capítulo são apresentadas as informações da disciplina de acordo com o estabelecido no **Projeto Pedagógico do Curso de Bacharelado em Sistemas de Informação** (PPC) de novembro de 2016.

Suas informações básicas são:

- **Período:** 4º.
- **Pré-requisito:** Programação Orientada a Objetos I.
- **Créditos:** 1.3.0.
- **Carga Horária**
 - Semanal: 4h.
 - Semestral: 60h.

1.1 Ementa

- Interfaces e processamento de eventos.
- Programação gráfica.
- Manipulação de Arquivos.
- Programação concorrente usando Multithreading.
- Programação em rede.
- Conectividade com bancos de dados: JDBC.

1.2 Bibliografia

1.2.1 Básica

1. BARNES, D. J.; KÖLLING, M. **Programação Orientada a Objetos com Java: Uma introdução prática usando BlueJ**. 4. ed. São Paulo: Pearson Prentice Hall, 2009.
2. DEITEL, Paul; DEITEL, Harvey. **Java: como programar**. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

3. FREEMAN, Eric; FREEMAN, Elisabeth. **Use a cabeça!: padrões de projetos**. 2. ed. Rio de Janeiro: Alta Books, 2007.

1.2.2 Complementar

4. SIERRA, Kathy; BATES, Bert. **Use a cabeça!: Java**. 1. ed. Rio de Janeiro: Alta Books, 2005.
5. HORSTMANN, Cay S.; CORNELL, Gary. **Core Java 2 Vol. 1 Fundamentos**. 7. ed. Rio de Janeiro: Alta Books, 2005.
6. KURNIAWAN, Budi. **Java para a Web com Servlets, JSP e EJB**. 1. ed. Rio de Janeiro: Ciência Moderna, 2002.
7. CADENHEAD, Rogers; LEMAY, Laura. **Aprenda em 21 dias Java 2**. 4. ed. Rio de Janeiro: Elsevier, 2005.
8. HORSTMANN, Cay S. **Big Java**. 4. ed. [S. l.]: John Wiley & Sons, 2006.

Capítulo 2

Informações atualizadas

A área de computação e suas correlatas — e.g., Sistemas de Informação, Análise e Desenvolvimento de Sistemas, etc. — são muito dinâmicas, i.e., em um espaço de poucos anos vários de seus conteúdos e objetos de estudo mudam ou são bastante, e até radicalmente atualizados.

Como consequência dessa dinamicidade, tanto a ementa quanto a bibliografia registradas no PPC podem ficar defasados em pouco tempo. Além disso, como se trata de um documento extenso e denso, sua atualização não é um trabalho trivial. Por exemplo, é necessário que uma equipe de professores compare as realidades anterior e contemporânea, tanto sob uma ótica acadêmica quanto sob uma de mercado e a partir disso compreender quais mudanças ocorreram nas demandas como um todo.

Em posse desse novo entendimento os docentes poderão discutir entre si para decidirem quais atualizações serão necessárias, ou seja, quais modificações deverão ser executadas de fato. A diferença de tempo, uma década (de 2016 a 2026), já é suficiente para que se preveja a necessidade de atualizações profundas. Ao mesmo tempo, parte do PPC pode ter sido pensado tendo em vista a dinamicidade da área de forma que, mesmo após tantos anos, muitos de seus tópicos não precisem ser atualizados, ou sofrer atualização mínima.

O *ementário dos componentes curriculares e bibliografia* é um dos tópicos do PPC que possivelmente sofre com a defasagem de maneira mais rápida. Suponha um PPC completamente atualizado ao fim de 2026. É possível que já em 2027 a ementa e bibliografia de algumas disciplinas já necessitem de atualização.

Diante disso é perfeitamente natural que os professores ajustem essas informações ao longo dos períodos, distanciando-se do previsto no PPC. Ao mesmo tempo, a própria abordagem metodológica também precisa ser ajustada, muitas vezes a um ritmo de ajuste semestral, considerando as evoluções de tecnologia que ocorrem de maneira tão rápida.

2.1 Planos de ensino anteriores

De início, como pode ser consultado no PPC, a ferramenta mais “básica” da disciplina consistia na linguagem de programação Java, uma vez que se trata de uma excelente tecnologia para a aprendizagem de conceitos relacionados à Orientação a Objetos (OO). Entretanto, nos semestres mais recentes, as turmas têm preferido a utilização da linguagem de programação Python.

A ementa tem sido mantida como base para a programação dos conteúdos a serem estudados, e as adaptações consistiram, em sua maior parte, na escolha de ferramentas e tecnologias que poderiam ser estudadas junto à linguagem Python. Por exemplo, considerando-se os dois primeiros tópicos da ementa, foram estudados nos semestres anteriores as bibliotecas Tkinter e PySide6, ambas bibliotecas que permitem a criação de interfaces gráficas para *desktop*, e o processamento de eventos a partir da interação de um usuário com seus componentes gráficos. Tanto a construção dos componentes quanto a manipulação dos eventos disparados são feitos com a aplicação direta de conceitos de OO.

Em seguida as turmas foram levadas ao estudo da manipulação de arquivos e *threads*, culminando na conexão de aplicativos feitos para *desktop* com bancos de dados remotos. Novamente, tudo sendo feito com a aplicação de conceitos de OO.

Uma vez que esta disciplina possui uma carga horária de 60 horas, ela necessita de três avaliações, as quais seguiam, de forma geral, o seguinte esquema cumulativo:

- **Avaliação 1:** prova escrita e o desenvolvimento de uma interface gráfica para uma aplicação *desktop*.
- **Avaliação 2:** continuação do desenvolvimento do aplicativo *desktop*, para abranger a aplicação dos quatro primeiros tópicos da ementa.
- **Avaliação 3:** finalização do desenvolvimento do aplicativo *desktop*, de forma que todos os tópicos da ementa tenham sido aplicados.

Contudo, devido à rápida evolução da IA, o seu uso ficou bastante notório, permitindo aos alunos a construção de interfaces gráficas consideravelmente complexas e também a integração dos demais tópicos da ementa sem que houvesse grandes desafios.

É importante ressaltar que os alunos não foram proibidos de utilizarem as ferramentas proporcionadas pela IA, ou seja, seu uso notório não estava ocorrendo à revelia. O propósito geral dessa permissão era permitir aos alunos começarem a ambientar-se ao uso de IA no desenvolvimento de suas aplicações, uma vez que essa será sua realidade no mercado de trabalho de desenvolvimento de software.

Por outro lado isso revelou que tal abordagem estava fazendo com que a disciplina cursada na universidade comesse a ficar esvaziada de sentido. Em outras palavras, tanto os objetos de estudo quanto as aplicações e avaliações estavam seguindo um esquema que pode ser facilmente encontrado na Internet, seja através de cursos gratuitos e/ou pagos, seja através de vídeos e tutoriais, além dos materiais de referência

fornecidos nos sites oficiais das tecnologias utilizadas.

Diante disso, a partir desse semestre será proposta uma abordagem diferente do que vinha sendo implementado até então. O foco será retirado do uso das diversas ferramentas disponíveis para a implementação de aplicações *desktop* com acesso remoto a um banco de dados, para um estudo mais aprofundado dos conceitos inerentes de cada tópico da ementa. Em outras palavras, o foco deixará de ser mais voltado ao mercado para se tornar mais voltado à academia.

2.2 Novo Plano de Ensino

A ementa continuará a mesma do PPC (com seus devidos ajustes), porém com objetivos diferentes daqueles dos planos de ensino anteriores. Nas subseções a seguir serão listados os novos subtópicos, objetivos e bibliografias para cada tópico da ementa.

2.2.1 Novos subtópicos da ementa

- Interfaces e processamento de eventos
 - Estudo do padrão *Observer* como fundamento teórico do processamento de eventos.
 - Desacoplamento entre emissor e receptor (inversão de dependência).
 - Modelagem de um *event loop*.
 - Comparação entre modelos: *callbacks*, *listeners*, *signals/slots* e eventos tipados.
 - Análise do *trade-off* entre tipagem forte de eventos e flexibilidade.
- Programação gráfica
 - Widgets e componentes como objetos compostos (*Composite pattern*).
 - Separação de responsabilidades: modelos de dados × apresentação × controle (MVC, MVP, MVVM).
 - Ciclo de vida de um componente gráfico e gerenciamento de estado encapsulado.
 - Herança versus composição na construção de interfaces customizadas.
 - O conceito de *scene graph*/árvore de objetos visuais e como a hierarquia de objetos se relaciona com a renderização.
 - Princípios de *layout* e *constraint solving* como problemas de modelagem.
- Manipulação de Arquivos.
 - Arquivos e *streams* como abstrações de recursos (*Resource Acquisition Is Initialization* — *RAII*).
 - Polimorfismo de *streams* (leitura de arquivo, memória, rede, etc. sob a mesma interface).
 - Gerenciamento de ciclo de vida e o problema de *ownership* de recursos.
 - Estratégias de *buffering* e o impacto no design das classes.

- Separação entre “acesso físico” e “interpretação de conteúdo” (ex.: File × Parser × Domain Object).
- Tratamento de erros como parte do contrato da interface (exceções vs. códigos de erro vs. tipos Result).
- Programação concorrente com múltiplas threads
 - Thread como objeto e as consequências de se ter estado compartilhado entre objetos.
 - Modelos de concorrência: memória compartilhada × troca de mensagens.
 - Princípios de sincronização: exclusão mútua, condições de corrida, deadlock e starvation.
 - Padrões clássicos: Producer-Consumer, Thread Pool, Future/Promise, Actor.
 - Imutabilidade e objetos *thread-safe* por design.
 - A diferença entre concorrência e paralelismo vista sob a ótica de responsabilidades de classes.
- Programação em rede
 - Socket e conexão como objetos com ciclo de vida bem definido.
 - Separação entre protocolo de transporte e protocolo de aplicação.
 - Padrões de arquitetura: Reactor, Proactor, Client-Server e Peer-to-Peer.
 - Serialização e o problema de representar objetos em formato de transmissão (object serialization vs. data transfer objects).
 - Tratamento de falhas de rede como parte do contrato das interfaces (timeouts, reconexão, idempotência).
 - Modelagem de sessões e estado de conexão.
- Conectividade com bancos de dados
 - O object-relational impedance mismatch como problema central.
 - Padrões de mapeamento: Active Record × Data Mapper × Repository × Unit of Work.
 - Transações como abstração de consistência e o princípio ACID visto sob a ótica de objetos.
 - Identity Map e o problema de identidade de objetos (quando dois objetos representam a mesma entidade do banco).
 - Lazy loading, eager loading e o impacto no design das classes de domínio.
 - Separação entre modelo de domínio e modelo de persistência.

A aparente quantidade “exaustiva” de subtópicos por enquanto estará servindo como uma meta. Em outras palavras, a sua introdução na disciplina será gradual, de acordo com a avaliação do acompanhamento das novas turmas, e também com a aquisição de experiência. De início serão estudados dois a três subtópicos de cada tópico da ementa, objetivando-se seu estudo mais profundo e, havendo tempo ou se a turma conseguir render bem, outro subtópico será introduzido.

Ao mesmo tempo os subtópicos listados não necessariamente deverão permanecer intactos, pois à medida em que se aprofunda a experiência tanto do docente quanto dos discentes, conjugado aos fatores externos (e.g., novas tecnologias e demandas),

essa listagem certamente sofrerá adaptações.

2.2.2 Novos objetivos

- Interfaces e processamento de eventos
 - Implementar um sistema de eventos genérico sem usar nenhuma biblioteca de GUI.
 - Modelar a diferença entre “evento síncrono” e “evento assíncrono” em termos de responsabilidades de classes.
 - Estudar como a inversão de controle aparece naturalmente no tratamento de eventos.
- Programação gráfica
 - Criar uma mini-biblioteca de componentes gráficos puramente em memória (sem desenho real) para estudar composição e notificação de mudanças.
 - Analisar por que a herança profunda de widgets costuma ser problemática e como a composição resolve isso.
- Manipulação de arquivos
 - Projetar uma hierarquia de classes de “fonte de dados” que possa ser tanto arquivo quanto memória ou rede, sem o cliente saber a diferença.
 - Estudar o padrão Decorator aplicado a streams (compressão, criptografia, buffering).
- Programação concorrente com múltiplas threads
 - Implementar um pool de threads e um mecanismo de futures sem usar bibliotecas prontas de alto nível.
 - Modelar um problema clássico (ex.: leitores-escritores) usando apenas interfaces e classes, e depois analisar onde a sincronização é realmente necessária.
 - Comparar o design orientado a objetos de uma solução com locks versus uma solução com canais/mensagens.
- Programação em rede
 - Projetar uma interface de “Canal de Comunicação” que abstrai TCP, UDP ou até comunicação em memória.
 - Implementar um protocolo de aplicação simples e estudar como a hierarquia de classes reflete as camadas do protocolo.
 - Analisar o padrão Reactor e sua relação com o event loop já estudado no tópico de eventos.
- Conectividade com bancos de dados
 - Implementar um repositório genérico e um Unit of Work simples em memória, depois adaptá-lo para um banco real.
 - Estudar por que a herança de tabelas (table inheritance) é problemática e quais alternativas de modelagem existem.
 - Analisar o ciclo de vida de um objeto de domínio desde o carregamento até a persistência, identificando pontos de acoplamento.

Da mesma forma que os subtópicos da ementa, esses objetivos listados servem como meta. Por enquanto somente um objetivo de cada tópico da ementa será proposto para as turmas e, havendo tempo ou se a turma conseguir evoluir bem, outro objetivo será proposto.

Outra característica similar à da listagem dos subtópicos é que tais objetivos são mutáveis. À medida da aquisição de experiência do docente e das respostas das turmas, os objetivos que existem podem ser adaptados, retirados, e outros novos podem surgir.

2.2.3 Nova bibliografia

A seguir serão listados os materiais que servirão de **Bibliografia Básica** e **Bibliografia Complementar** para cada tópico da ementa.

2.2.3.1 Interfaces e processamento de eventos

- **Bibliografia Básica**

- FREEMAN, Eric; ROBSON, Elisabeth. **Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software**. 2. ed. [S. l.]: O'Reilly Media, 2020.
- HORSTMANN, Cay S. **Object-Oriented Design and Patterns**. 2. ed. [S. l.]: John Wiley & Sons, 2005
- GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S. l.]: Addison-Wesley, 1995.

- **Bibliografia Complementar**

- GURU, Refactoring. **Design Patterns**. Disponível em: <https://refactoring.guru/design-patterns/>. Acesso em: 30 jul. 2026.
- OODESIGN. **Design Patterns**. Disponível em: <https://www.oodeesign.com/>. Acesso em: 30 jul. 2026.
- OKHRAVI, Christopher. **Design Patterns in Object Oriented Programming**. 2020. Disponível em: <https://www.youtube.com/playlist?list=PLrhzvIcii6GNjpARdn04ueTUAVR9eMBpc>. Acesso em: 30 jul. 2026.
- BANAS, Derek. **Design Patterns Video Tutorial**. 2014. Disponível em: <https://www.youtube.com/playlist?list=PLF206E906175C7E07>. Acesso em: 30 jul. 2026.

O livro-texto para esse tópico será o livro (Freeman; Robson, 2020), uma vez que se trata do mais recente. Os livros (Horstmann, 2005) e (Gamma *et al.*, 1995) servirão de apoio, caso haja necessidade. Esse último é uma referência clássica.

Exemplos de código podem ser encontrados em (Guru, 2026), e as referências (Okhravi, 2020; Banas, 2014) consistem em *playlists* sobre Design Patterns no YouTube.

2.2.3.2 Programação gráfica

- **Bibliografia Básica**

- TIDWELL, Jenifer; BREWER, Charles; VALENCIA, Aynne. **Designing Interfaces: Patterns for Effective Interaction Design**. 3. ed. [S. l.]: O'Reilly Media, 2020.
- **Bibliografia Complementar**
 - HORSTMANN, Cay S. **Object-Oriented Design and Patterns**. 2. ed. [S. l.]: John Wiley & Sons, 2005
 - GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S. l.]: Addison-Wesley, 1995.

O livro (Tidwell; Brewer; Valencia, 2020) será o livro-texto desse tópico, acompanhado das referências (Gamma *et al.*, 1995; Horstmann, 2005). Além do padrão Observer, estudado no primeiro tópico, neste ponto é interessante estudar também outros padrões como Composite e Decorator, além da Arquitetura MVC.

2.2.3.3 Manipulação de arquivos

- **Bibliografia Básica**
 - FOLK, Michael J.; ZOELLICK, Bill; RICCARDI, Greg. **File Structures: An Object-Oriented Approach with C++**. [S. l.]: Addison-Wesley, 1998.
- **Bibliografia Complementar**
 - DEV.JAVA. **The Java I/O API**. disponível em: <https://dev.java/learn/java-io/>. Acesso em: 30 jul. 2026.
 - DEV.JAVA. **Common I/O Tasks in Modern Java**. Disponível em: <https://dev.java/learn/modernio/>. Acesso em: 30 jul. 2026.
 - DEV.JAVA. **The Stream API**. disponível em: <https://dev.java/learn/api/streams/>. Acesso em: 30 jul. 2026.
 - ORACLE. **Java SE 25 Core Libraries: Java NIO**. disponível em: <https://docs.oracle.com/en/java/javase/25/core/java-nio.html>. Acesso em: 30 jul. 2026.
 - ARYANNAYAKK. **Understanding the Hierarchy of Java Stream**. 2023. Disponível em: <https://medium.com/@aryannayakk/hierarchy-of-java-streams-45b3badc4682>. Acesso em: 30 jul. 2026.
 - FOUNDATION, Python Software. **io — Core tools for working with streams**. Disponível em: <https://docs.python.org/3/library/io.html>. Acesso em: 30 jul. 2026.

Apesar de a referência principal ser de 1998, o objetivo nesta parte do curso será estudar de modo aprofundado como manipular arquivos que, de forma “genérica”, consiste na manipulação de streams. A partir de (Folk; Zoellick; Riccardi, 1998) procuraremos aprender os conceitos mais fundamentais, enquanto a prática poderá ocorrer com a utilização de Java, seguindo a bibliografia complementar, ou com o uso de Python, de acordo com a última referência.

2.2.3.4 Programação concorrente com múltiplas threads

- **Bibliografia Básica**

- HERLIHY, Maurice *et al.* **The Art of Multiprocessor Programming**. 2. ed. [S. l.]: Morgan Kaufmann, 2020.

- **Bibliografia Complementar**

- GOETZ, Brian *et al.* **Java Concurrency in Practice**. [S. l.]: Addison-Wesley, 2006.
- DEV.JAVA. **Virtual Threads**. Disponível em: <https://dev.java/learn/new-features/virtual-threads/>. Acesso em: 30 jul. 2026.
- ORACLE. **Java SE 25 Core Libraries: Concurrency**. Disponível em: <https://docs.oracle.com/en/java/javase/25/core/concurrency.html>. Acesso em: 30 jul. 2026.
- FOUNDATION, Python Software. **threading — Thread-based parallelism**. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 30 jul. 2026.
- FOUNDATION, Python Software. **asyncio — Asynchronous I/O**. disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 30 jul. 2026.
- FOUNDATION, Python Software. **multiprocessing — Process-based parallelism**. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: 30 jul. 2026.
- SHEETS, Python. **Concurrency**. Disponível em: <https://www.pythonsheets.com/notes/concurrency/index.html>. Acesso em: 30 jul. 2026.

O livro (Herlihy *et al.*, 2020) será utilizado como base para o estudo sobre princípios, algoritmos e demais informações sobre concorrência e paralelismo. A prática ocorrerá com a utilização de módulos e bibliotecas nativas da linguagem de programação Java, ou da linguagem Python.

2.2.3.5 Programação em rede

- **Bibliografia Básica**

- HALL, Brian “Beej Jorgensen”. **Beej’s Guide to Network Programming**. Versão v3.3.2. 2026. Disponível em: https://beej.us/guide/bgnet/pdf/bgnet_a4_c_2.pdf. Acesso em: 30 jul. 2026.
- STEVENS, W. Richard; FENNER, Bill; RUDOFF, Andrew M. **UNIX Network Programming, Volume 1: The Sockets Networking API**. 3. ed. [S. l.]: Addison-Wesley, 2003.

- **Bibliografia Complementar**

- ORACLE. **Java SE 25 Core Libraries: Java Networking**. Disponível em: <https://docs.oracle.com/en/java/javase/25/core/java-networking.html#GUID-82A738CB-2A1C-4AC8-B9DA-C4543D398B51>. Acesso em: 30 jul. 2026.

- FOUNDATION, Python Software. **multiprocessing — Process-based parallelism**. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: 30 jul. 2026.

Novamente teremos os livros da bibliografia básica como fonte principal para estudo de bases e princípios, enquanto a prática ocorrerá com o uso de módulos ou bibliotecas nativas das linguagens Java ou Python.

2.2.3.6 Conectividade com bancos de dados

- **Bibliografia Básica**
- **Bibliografia Complementar**

Referências

ARYANNAYAKK. **Understanding the Hierarchy of Java Stream**. 2023. Disponível em: <https://medium.com/@aryannayakk/hierarchy-of-java-streams-45b3badc4682>. Acesso em: 30 jul. 2026. Citado na p. 11.

BANAS, Derek. **Design Patterns Video Tutorial**. 2014. Disponível em: <https://www.youtube.com/playlist?list=PLF206E906175C7E07>. Acesso em: 30 jul. 2026. Citado na p. 10.

BARNES, D. J.; KÖLLING, M. **Programação Orientada a Objetos com Java: Uma introdução prática usando BlueJ**. 4. ed. São Paulo: Pearson Prentice Hall, 2009. Citado na p. 3.

CADENHEAD, Rogers; LEMAY, Laura. **Aprenda em 21 dias Java 2**. 4. ed. Rio de Janeiro: Elsevier, 2005. Citado na p. 4.

DEITEL, Paul; DEITEL, Harvey. **Java: como programar**. 8. ed. São Paulo: Pearson Prentice Hall, 2010. Citado na p. 3.

DEV.JAVA. **Common I/O Tasks in Modern Java**. Disponível em: <https://dev.java/learn/modernio/>. Acesso em: 30 jul. 2026. Citado na p. 11.

DEV.JAVA. **The Java I/O API**. Disponível em: <https://dev.java/learn/java-io/>. Acesso em: 30 jul. 2026. Citado na p. 11.

DEV.JAVA. **The Stream API**. Disponível em: <https://dev.java/learn/api/streams/>. Acesso em: 30 jul. 2026. Citado na p. 11.

DEV.JAVA. **Virtual Threads**. Disponível em: <https://dev.java/learn/new-features/virtual-threads/>. Acesso em: 30 jul. 2026. Citado na p. 12.

FOLK, Michael J.; ZOELLICK, Bill; RICCARDI, Greg. **File Structures: An Object-Oriented Approach with C++**. [S. l.]: Addison-Wesley, 1998. Citado na p. 11.

FOUNDATION, Python Software. **asyncio — Asynchronous I/O**. Disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 30 jul. 2026. Citado na p. 12.

FOUNDATION, Python Software. **io — Core tools for working with streams**. Disponível em: <https://docs.python.org/3/library/io.html>. Acesso em: 30 jul. 2026. Citado na p. 11.

FOUNDATION, Python Software. **multiprocessing — Process-based parallelism**. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: 30 jul. 2026. Citado nas pp. 12, 13.

FOUNDATION, Python Software. **threading — Thread-based parallelism**. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 30 jul. 2026. Citado na p. 12.

FREEMAN, Eric; FREEMAN, Elisabeth. **Use a cabeça!: padrões de projetos**. 2. ed. Rio de Janeiro: Alta Books, 2007. Citado na p. 4.

FREEMAN, Eric; ROBSON, Elisabeth. **Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software**. 2. ed. [S. l.]: O'Reilly Media, 2020. Citado na p. 10.

GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S. l.]: Addison-Wesley, 1995. Citado nas pp. 10, 11.

GOETZ, Brian *et al.* **Java Concurrency in Practice**. [S. l.]: Addison-Wesley, 2006. Citado na p. 12.

GURU, Refactoring. **Design Patterns**. Disponível em: <https://refactoring.guru/design-patterns/>. Acesso em: 30 jul. 2026. Citado na p. 10.

HALL, Brian “Beej Jorgensen”. **Beej’s Guide to Network Programming**. Versão v3.3.2. 2026. Disponível em: https://beej.us/guide/bgnet/pdf/bgnet_a4_c_2.pdf. Acesso em: 30 jul. 2026. Citado na p. 12.

HERLIHY, Maurice *et al.* **The Art of Multiprocessor Programming**. 2. ed. [S. l.]: Morgan Kaufmann, 2020. Citado na p. 12.

HORSTMANN, Cay S. **Big Java**. 4. ed. [S. l.]: John Wiley & Sons, 2006. Citado na p. 4.

HORSTMANN, Cay S. **Object-Oriented Design and Patterns**. 2. ed. [S. l.]: John Wiley & Sons, 2005. Citado nas pp. 10, 11.

HORSTMANN, Cay S.; CORNELL, Gary. **Core Java 2 Vol. 1 Fundamentos**. 7. ed. Rio de Janeiro: Alta Books, 2005. Citado na p. 4.

KURNIAWAN, Budi. **Java para a Web com Servlets, JSP e EJB**. 1. ed. Rio de Janeiro: Ciência Moderna, 2002. Citado na p. 4.

OKHRAVI, Christopher. **Design Patterns in Object Oriented Programming**. 2020. Disponível em:
<https://www.youtube.com/playlist?list=PLrhzvIcii6GNjpARdn04ueTUAVR9eMBpc>. Acesso em: 30 jul. 2026. Citado na p. 10.

OODDESIGN. **Design Patterns**. Disponível em: <https://www.oodesign.com/>. Acesso em: 30 jul. 2026. Citado na p. 10.

ORACLE. **Java SE 25 Core Libraries: Concurrency**. Disponível em:
<https://docs.oracle.com/en/java/javase/25/core/concurrency.html>. Acesso em: 30 jul. 2026. Citado na p. 12.

ORACLE. **Java SE 25 Core Libraries: Java Networking**. Disponível em:
<https://docs.oracle.com/en/java/javase/25/core/java-networking.html#GUID-82A738CB-2A1C-4AC8-B9DA-C4543D398B51>. Acesso em: 30 jul. 2026. Citado na p. 12.

ORACLE. **Java SE 25 Core Libraries: Java NIO**. Disponível em:
<https://docs.oracle.com/en/java/javase/25/core/java-nio.html>. Acesso em: 30 jul. 2026. Citado na p. 11.

SHEETS, Python. **Concurrency**. Disponível em:
<https://www.pythonsheets.com/notes/concurrency/index.html>. Acesso em: 30 jul. 2026. Citado na p. 12.

SIERRA, Kathy; BATES, Bert. **Use a cabeça!: Java**. 1. ed. Rio de Janeiro: Alta Books, 2005. Citado na p. 4.

STEVENS, W. Richard; FENNER, Bill; RUDOFF, Andrew M. **UNIX Network Programming, Volume 1: The Sockets Networking API**. 3. ed. [S. l.]: Addison-Wesley, 2003. Citado na p. 12.

TIDWELL, Jenifer; BREWER, Charles; VALENCIA, Aynne. **Designing Interfaces: Patterns for Effective Interaction Design**. 3. ed. [S. l.]: O'Reilly Media, 2020. Citado na p. 11.